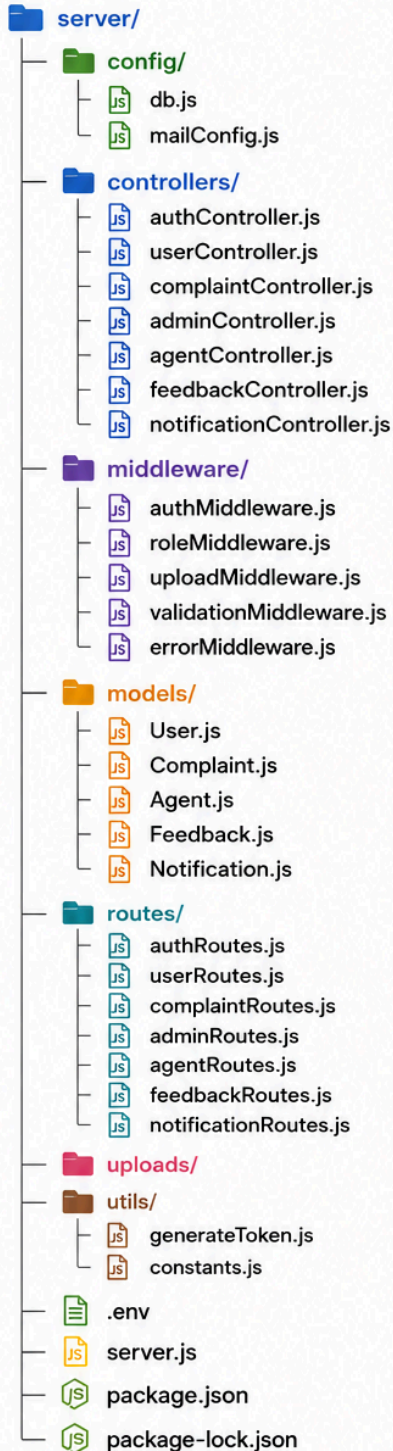


ONLINE COMPLAINT REGISTRATION – SERVER STRUCTURE



1. config/

- **db.js** – Database connection configuration (MongoDB).
- **mailConfig.js** – Email service configuration using Nodemailer.



2. controllers/

Contains business logic for the application.

- **authController.js** – Handles authentication & user login.
- **userController.js** – Manage user profile and dashboard.
- **complaintController.js** – Manage complaints (CRUD, status, assign).
- **adminController.js** – Admin operations and reports.
- **agentController.js** – Agent operations and complaint updates.
- **feedbackController.js** – Handle user feedback.
- **notificationController.js** – Send and manage notifications.



3. middleware/

- **authMiddleware.js** – Verify JWT and authenticate users.
- **roleMiddleware.js** – Check user roles (Admin, User, Agent).
- **uploadMiddleware.js** – Handle file uploads using Multer.
- **validationMiddleware.js** – Validate incoming request data.
- **errorMiddleware.js** – Handle errors and send error responses.



4. models/

- **User.js** – User schema and model.
- **Complaint.js** – Complaint schema and model.
- **Agent.js** – Agent schema and model.
- **Feedback.js** – Feedback schema and model.
- **Notification.js** – Notification schema and model.



5. routes/

- **authRoutes.js** – Authentication related routes.
- **userRoutes.js** – User related routes.
- **complaintRoutes.js** – Complaint related routes.
- **adminRoutes.js** – Admin related routes.
- **agentRoutes.js** – Agent related routes.
- **feedbackRoutes.js** – Feedback related routes.
- **notificationRoutes.js** – Notification related routes.



6. uploads/

Stores uploaded files like complaint images, documents, and evidence.



7. utils/

- **generateToken.js** – Generate JWT tokens.
- **constants.js** – Application wide constants (roles, status, messages).



8. Root Files

- **.env** – Environment variables (DB URL, JWT Secret, Email, etc.).
- **server.js** – Entry point, start server, connect DB, load routes.
- **package.json** – Project info and dependencies.
- **package-lock.json** – Lock file for exact dependency versions.

WORKFLOW



This modular structure follows MVC architecture, making the backend organized, maintainable, scalable and easier to develop and test.